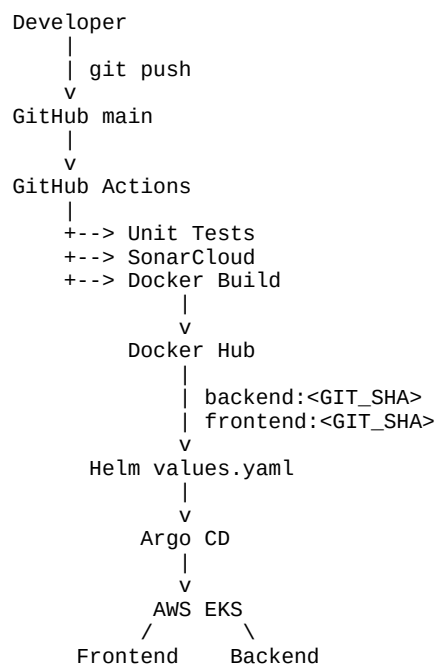


# GitOps with Argo CD on AWS EKS

Step-by-step implementation guide for a MERN E-commerce application using GitHub Actions, Docker Hub, Helm, Argo CD, and AWS EKS.

**Project approach:** GitHub *main* is the source of truth. GitHub Actions builds immutable Docker images using the Git commit SHA. Helm defines the Kubernetes desired state, and Argo CD reconciles that desired state into EKS.

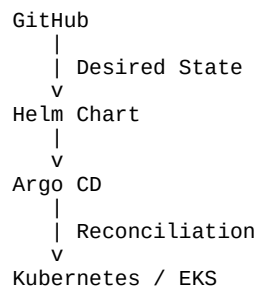


# Implementation Roadmap

| Step    | Task                           |
|---------|--------------------------------|
| Step 1  | Connect kubectl to EKS         |
| Step 2  | Create Argo CD namespace       |
| Step 3  | Install Argo CD                |
| Step 4  | Verify Argo CD pods            |
| Step 5  | Access Argo CD UI              |
| Step 6  | Get Argo CD admin password     |
| Step 7  | Install Argo CD CLI            |
| Step 8  | Inspect / verify Helm chart    |
| Step 9  | Create Argo CD Application     |
| Step 10 | Connect GitHub main to Argo CD |
| Step 11 | Initial Argo CD sync           |
| Step 12 | Verify application on EKS      |
| Step 13 | Test GitOps deployment         |
| Step 14 | Test rollback                  |

## GitOps Concept

GitOps treats Git as the source of truth for the desired application state. Argo CD continuously compares the desired state in Git with the actual state in Kubernetes and reconciles differences.



## Project Structure

```
mern-ecommerce-k8s-github-actions/
|
+-- .github/
|   +-- workflows/
|       +-- ci-cd.yml
|
+-- backend/
|
+-- frontend/
|
+-- mern-ecommerce/
|   +-- Chart.yaml
|   +-- values.yaml
|   +-- templates/
|
+-- README.md
```

## Step 1 — Connect kubectl to AWS EKS

Verify AWS authentication, configure kubeconfig for the EKS cluster, and confirm that kubectl can see the worker nodes.

### Check AWS credentials

```
aws sts get-caller-identity
```

### Configure kubeconfig

```
aws eks update-kubeconfig \
  --region <YOUR_REGION> \
  --name <YOUR_EKS_CLUSTER_NAME>
```

### Verify the connection

```
kubectl get nodes
```

Expected: your EKS nodes should normally show STATUS as **Ready**.

## Step 2 — Create Argo CD Namespace

Use a dedicated namespace to keep Argo CD components separate from the application workloads.

```
kubectl create namespace argocd
kubectl get namespace argocd
EKS Cluster
|
+-- argocd
|   +-- argocd-server
|   +-- argocd-repo-server
|   +-- argocd-application-controller
|   +-- argocd-redis
|
+-- mern-ecommerce
    +-- frontend
    +-- backend
```

## Step 3 — Install Argo CD

```
kubectl apply -n argocd \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

This installs the core Argo CD components into the argocd namespace.

## Step 4 — Verify Argo CD Pods

```
kubectl get pods -n argocd
kubectl get pods -n argocd -w
```

Wait until the important Argo CD pods are Running and Ready. Press Ctrl+C to stop watching.

## Step 5 — Access Argo CD UI

```
kubectl port-forward svc/argocd-server \
  -n argocd 8080:443
```

Open **https://localhost:8080** in your browser. The initial Argo CD server uses a self-signed certificate, so a browser warning may appear.

## Step 6 — Get Argo CD Admin Password

```
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d
```

Username: **admin**. Use the password returned by the command to log in to the UI.

## Step 7 — Install Argo CD CLI

```
argocd version --client
argocd login localhost:8080 --insecure
argocd account get-user-info
```

The CLI is useful for inspecting applications, syncing, viewing history, and performing operational tasks.

## Step 8 — Inspect and Verify Helm Chart

The Helm chart used by this project is located at [mern-ecommerce/](#).

```
mern-ecommerce/
|
+-- Chart.yaml
|
+-- values.yaml
|
+-- templates/
    +-- backend-deployment.yaml
    +-- frontend-deployment.yaml
    +-- backend-service.yaml
    +-- frontend-service.yaml
    +-- ...
```

### values.yaml — immutable image tags

```
backend:
  image:
    repository: vipulsingh2200/mern-ecommerce-backend
    tag: "GIT_SHA"
  replicas: 3

frontend:
  image:
    repository: vipulsingh2200/mern-ecommerce-frontend
    tag: "GIT_SHA"
  replicas: 3
```

### Deployment image references

```
# Backend
image: "{{ .Values.backend.image.repository }}:{{ .Values.backend.image.tag }}"

# Frontend
image: "{{ .Values.frontend.image.repository }}:{{ .Values.frontend.image.tag }}"
```

### Validate the chart

```
helm lint ./mern-ecommerce
helm template mern-ecommerce ./mern-ecommerce
```

helm lint should report no chart failures. helm template renders manifests without deploying them.

## Step 9 — Create Argo CD Application

Create a directory and application manifest that tells Argo CD which Git repository, branch, and Helm chart to monitor.

```
mkdir argocd
touch argocd/application.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application

metadata:
  name: mern-ecommerce
  namespace: argocd

spec:
  project: default

  source:
    repoURL: https://github.com/<YOUR_USERNAME>/<YOUR_REPOSITORY>.git
    targetRevision: main
    path: mern-ecommerce

  destination:
    server: https://kubernetes.default.svc
```

```

namespace: mern-ecommerce

syncPolicy:
  automated:
    prune: true
    selfHeal: true

```

## Step 10 — Connect GitHub main to Argo CD

The critical source configuration is:

```

source:
  repoURL: https://github.com/<YOUR_USERNAME>/<YOUR_REPOSITORY>.git
  targetRevision: main
  path: mern-ecommerce
GitHub Repository
  |
  v
  main
  |
  v
  mern-ecommerce/
  |
  v
  Helm Chart
  |
  v
  Argo CD
  |
  v
  EKS

```

Argo CD watches the main branch and uses the mern-ecommerce directory as the application source.

## Step 11 — Initial Argo CD Sync

```

kubectl apply -f argocd/application.yaml

kubectl get applications -n argocd

argocd app list

argocd app get mern-ecommerce

```

If the application is OutOfSync, perform the first manual sync:

```

argocd app sync mern-ecommerce
argocd app get mern-ecommerce

```

Expected end state: Sync Status = Synced and Health Status = Healthy.

## Step 12 — Verify Application on EKS

```

kubectl get namespace
kubectl get pods -n mern-ecommerce
kubectl get deployments -n mern-ecommerce
kubectl get svc -n mern-ecommerce
kubectl get all -n mern-ecommerce

```

The expected workload is a frontend deployment, a backend deployment, their services, and the configured replica pods.

## Step 13 — Test GitOps Deployment

This test proves that a Git change becomes a Kubernetes change through Argo CD.

### 13.1 Push application code

```

git add .
git commit -m "Update application"
git push origin main

```

### 13.2 GitHub Actions builds and pushes images

The CI pipeline runs unit tests, SonarCloud, Docker builds, and pushes both latest and immutable Git-SHA tags to Docker Hub.

```
vipulsing2200/mern-ecommerce-backend:<GIT_SHA>
vipulsing2200/mern-ecommerce-frontend:<GIT_SHA>
```

### 13.3 Manually update Helm values.yaml

For this learning setup, image tags are updated manually after a successful image build.

```
backend:
  image:
    repository: vipulsingh2200/mern-ecommerce-backend
    tag: "NEW_GIT_SHA"
  replicas: 3

frontend:
  image:
    repository: vipulsingh2200/mern-ecommerce-frontend
    tag: "NEW_GIT_SHA"
  replicas: 3
```

### 13.4 Commit the Helm change

```
git add mern-ecommerce/values.yaml
git commit -m "Update application image"
git push origin main
```

### 13.5 Verify Argo CD reconciliation

```
argocd app get mern-ecommerce
kubectl get pods -n mern-ecommerce
```

### 13.6 Verify the running image

```
kubectl get pods -n mern-ecommerce \
-o jsonpath="{.items[*].spec.containers[*].image}"
```

The running workloads should reference the new Git-SHA image tags.

## Step 14 — Test Rollback

Argo CD maintains application history so previous desired states can be reviewed and restored.

```
argocd app history mern-ecommerce
```

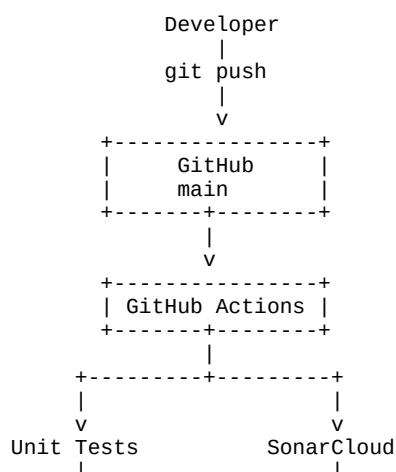
If a revision is known to be good, rollback to it:

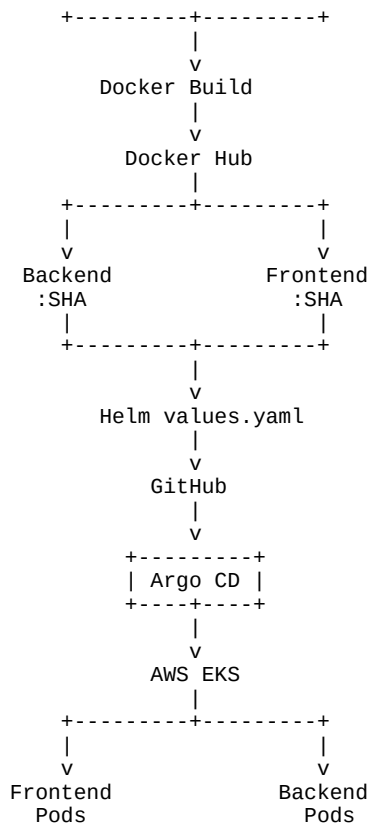
```
argocd app rollback mern-ecommerce <REVISION>
```

```
argocd app get mern-ecommerce
kubectl get pods -n mern-ecommerce
```

Always verify application health and workload readiness after a rollback.

## Final GitOps Architecture





## Key GitOps Principles

- GitHub is the source of truth for desired state.
- Helm packages and parameterizes the Kubernetes application.
- Argo CD continuously reconciles Git desired state with EKS actual state.
- Docker images use immutable Git-SHA tags rather than relying on latest for deployment.
- Kubernetes runs the actual application workload.
- Rollback is possible through Argo CD application history.

## Quick Command Reference

### AWS / EKS

```
aws sts get-caller-identity

aws eks update-kubeconfig \
  --region <REGION> \
  --name <CLUSTER_NAME>

kubectl get nodes
```

### Argo CD

```
kubectl get pods -n argocd

argocd app list

argocd app get mern-ecommerce

argocd app sync mern-ecommerce

argocd app history mern-ecommerce
```

### Kubernetes

```
kubectl get pods -n mern-ecommerce
```

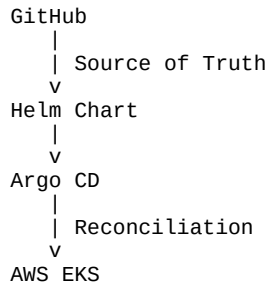
```
kubectrl get deployments -n mern-ecommerce
kubectrl get svc -n mern-ecommerce
kubectrl get all -n mern-ecommerce
```

## Helm

```
helm lint ./mern-ecommerce
helm template mern-ecommerce ./mern-ecommerce
```

## Conclusion

The completed workflow is:



**Git = Desired State**

**Argo CD = Reconciliation**

**EKS = Actual State**